FIAP · FASE 7

Smart HAS

Consolidação em Oracle PL/SQL e automação com CI/CD no GitHub Actions

GitHub: github.com/GabrielGaruti/smarthas

Vídeo: https://youtu.be/1oKfmJUCh9Y

INTEGRANTES



Gabriel Garuti Paiva Cracco
RM 554866



Vitor Cremonezi Morija
RM 550998

DO PROTÓTIPO AO PRODUTO

O que a Fase 7 entrega

4

packages PL/SQL

substituem as 8 rotinas avulsas da Fase 6

3

triggers

auditoria de alertas, status automático, validação IoT

2

rotinas de SQL dinâmico

busca com filtros e GROUP BY em tempo real

4

jobs de CI/CD

build, testes, Oracle efêmero, deploy

36 testes automatizados

28 no H2 (JUnit, MockMvc) + 8 com Oracle real a cada push

Deploy verificado

Imagem Docker no GHCR, API em staging com Oracle e smoke test de 11 verificações

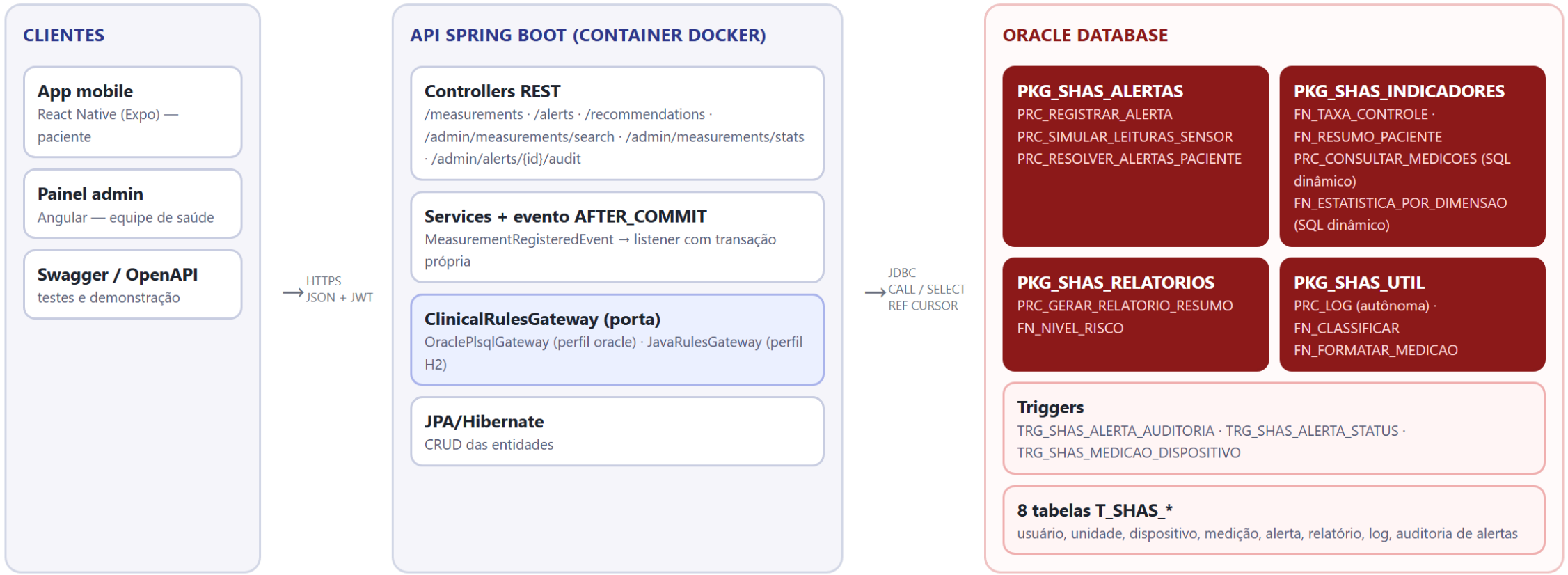
Um bug antigo corrigido

Os testes acharam 141x85 classificado como "Elevada" desde as primeiras fases

ARQUITETURA

Uma API, dois clientes, regras no Oracle

Arquitetura do Smart HAS na Fase 7



PARTE 1 · PACKAGES

8 rotinas avulsas viraram 4 packages

PKG_SHAS_UTIL

PRC_LOG · FN_CLASSIFICAR · FN_FORMATAR_MEDICAO · meta 140/90

PKG_SHAS_INDICADORES

FN_TAXA_CONTROLE · FN_RESUMO_PACIENTE · PRC_CONSULTAR_MEDICOES · FN_ESTATISTICA_POR_DIMENSAO

PKG_SHAS_ALERTAS

PRC_REGISTRAR_ALERTA · PRC_SIMULAR_LEITURAS_SENSOR · PRC_RESOLVER_ALERTAS_PACIENTE

PKG_SHAS_RELATORIOS

PRC_GERAR_RELATORIO_RESUMO · FN_NIVEL_RISCO

Por que packages

- **Encapsulamento:** rotinas privadas só no body
- **Uma fonte de verdade:** a meta 140/90 é constante única
- **Menos invalidação:** mudar o body não quebra quem usa a spec
- **Desempenho:** package carregado em memória na 1ª chamada
- **Integração:** o Java chama `PKG_SHAS_ALERTAS.PRC_REGISTRAR_ALERTA` via SimpleJdbcCall

PARTE 1 · TRIGGERS

Auditoria e automação no próprio banco

TRG_SHAS_ALERTA_AUDITORIA

AFTER INSERT/UPDATE/DELETE: grava operação, status anterior e novo, usuário e programa de origem em T_SHAS_AUDITORIA_ALERTA. Alerta CRÍTICO vai para o log.

TRG_SHAS_ALERTA_STATUS

BEFORE UPDATE OF ST_ALERTA: preenche a data de resolução ao resolver e limpa ao reabrir. Quem altera só muda o status.

TRG_SHAS_MEDICAO_DISPOSITIVO

BEFORE INSERT ... WHEN: só aceita aparelho ativo e do próprio paciente, marca SENSOR e atualiza a última sincronização.

Por que o alerta não é uma trigger?

A regra lê as últimas leituras da própria T_SHAS_MEDICAO: numa trigger de linha isso daria **ORA-04091 (tabela mutante)**. O alerta segue no evento de back-end AFTER_COMMIT, que chama o package; as triggers ficam com auditoria, datas e integridade.

PARTE 1 · DYNAMIC SQL

Filtros combinados em tempo real, sem SQL injection

```
-- PRC_CONSULTAR_MEDICOES: so os filtros informados
add_filtro(v_classif IS NOT NULL,
  'FN_CLASSIFICAR(...) = :b_classif', ':b_classif');
-- ORDER BY por whitelist
v_order := CASE p_ordenar_por
  WHEN 'SISTOLICA' THEN 'm.VL_SISTOLICA' ... END;
EXECUTE IMMEDIATE 'SELECT COUNT(*)' || v_from || v_where
  INTO p_total USING ...;
OPEN p_cursor FOR p_sql USING ...;

-- SQL gerado (capturado no pipeline):
 WHERE 1 = 1
  AND (1 = 1 OR :b_usuario IS NULL)
  AND PKG_SHAS_UTIL.FN_CLASSIFICAR(m.VL_SISTOLICA, m.VL_DIASTOLICA) =
:b_classif
  AND (1 = 1 OR :b_origem IS NULL)
  AND (1 = 1 OR :b_dias IS NULL)
  AND (1 = 1 OR :b_sis_min IS NULL)
 ORDER BY m.VL_SISTOLICA DESC, m.ID_MEDICAO DESC
 FETCH FIRST :b_limite ROWS ONLY
```

Binds sempre

Valores do usuário entram por USING; nomes de coluna só de uma lista fechada

GROUP BY dinâmico

FN_ESTATISTICA_POR_DIMENSAO: classificação, origem, paciente ou período do dia

Na API

GET /admin/measurements/search
GET /admin/measurements/stats

Teste de injeção

"DT_MEDICAO; DROP TABLE..." → ORA-20021

PARTE 2 · CI/CD

Pipeline no GitHub Actions

Pipeline CI/CD no GitHub Actions — .github/workflows/ci-cd.yml

GATILHO

push
main, fase7

pull request
para main

manual
workflow_dispatch

→

INTEGRAÇÃO CONTÍNUA (EM PARALELO)

backend

Build e testes

- mvn verify (Java 17)
- JUnit 5 + MockMvc + H2
- cobertura JaCoCo
- publica .jar e relatórios

web

Painel Angular

- npm ci
- ng build --configuration production

oracle-integration

Banco Oracle efêmero

- Oracle Database Free (container)
- run_all.sql: tabelas, packages, triggers, carga, testes SQL
- 99_validar_objetos.sql (falha se INVALID)
- testes Java → JDBC → PL/SQL

→

ENTREGA CONTÍNUA (MAIN / MANUAL)

deploy

Staging automatizado

- migração do banco de staging
- docker build da API
- push da imagem no GHCR
- API em container com perfil oracle
- smoke test (11 verificações)
- opcional: deploy_plsql.sql e hook de produção via secrets

→

LOGS E RELATÓRIOS

Summary
tabelas de testes, cobertura, instalação do banco, smoke test

Artefatos
logs de build, banco e API · Surefire · JaCoCo · .jar · dist

Badge
status no README

PARTE 2 · BUILD E TESTES

36 testes a cada push

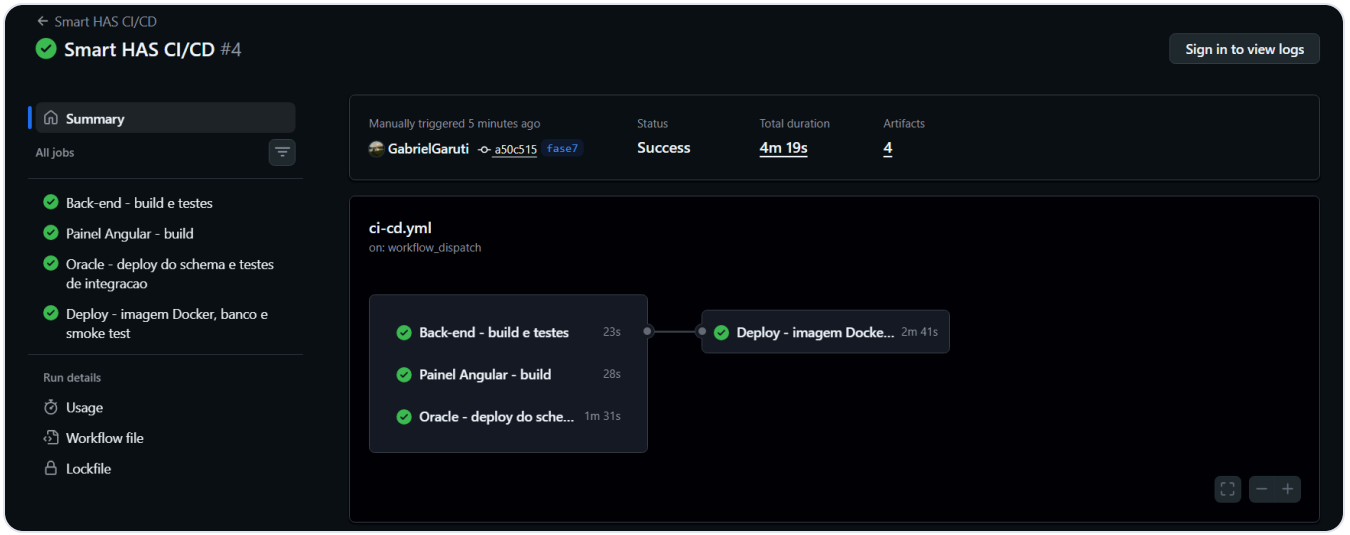
Classe	Testes	Cobre
ClassificationTest	9	limites da classificação
JavaRulesGatewayTest	11	alertas, busca dinâmica, relatório
ApiFlowTest	8	JWT, perfis, fluxos HTTP
OraclePlsqlGatewayIntegrationTest	8	packages, triggers, REF CURSOR no Oracle
Cobertura (JaCoCo)	**62,4%** das linhas no perfil H2	

O pipeline achou um bug antigo

141x85 era classificado como **Elevada**: a faixa 80–89 da diastólica era testada antes da sistólica ≥ 140.

Erro igual no Java e no PL/SQL desde as primeiras fases. Ordem corrigida nas duas camadas e caso virou teste permanente.

Execução #4: tudo verde



Deploy automatizado

run_all.sql no Oracle de staging → imagem no GHCR → API com perfil oracle → smoke test

Banco validado

99_validar_objetos.sql derruba o deploy se algum package ou trigger ficar INVALID

Logs e relatórios

Summary com testes, cobertura e smoke test; artefatos com logs do build, do banco e da API

Produção via secrets

deploy_plsql.sql no Oracle real e hook da API quando configurados

DEMONSTRAÇÃO E PRÓXIMOS PASSOS

O sistema rodando

1

App mobile

Paciente registra 186x121 e recebe o alerta CRÍTICO na hora

2

Swagger

Busca dinâmica, estatística por dimensão e trilha de auditoria

3

SQL Developer

08_testes_e_consultas.sql: packages, triggers e erros tratados

4

GitHub Actions

Push → jobs verdes → Summary e artefatos

Próximos passos

- Produção em nuvem ativando os secrets do pipeline
- Busca dinâmica e auditoria na interface do painel
- Relatório agendado com DBMS_SCHEDULER
- IA generativa explicando as leituras ao paciente

Obrigado!

github.com/GabrielGaruti/smarthas
Vídeo: https://youtu.be/1oKfmJUCh9Y

Gabriel Garuti Paiva Cracco — RM 554866
Vitor Cremonezi Morija — RM 550998